Practice Questions OOP (LAB) Dated: 02-07-2026

Q1. Static Fields & Static Methods

Write a C++ class `Counter` that has:

- A **static integer field** `count` initialized to 0.
- A **static method** `increment()` that increases `count` by 1.
- A **static method** `getCount()` that returns the current value of `count`.
  In `main()`, call `increment()` three times, then print the result of `getCount()`.

Q2. Method Signatures & Multiple Parameters

Write **two overloaded functions** named `multiply`:

- One takes **two integers** and returns their product.
- The other takes **three integers** and returns their product.
  In `main()`, call both functions with suitable arguments and print the results.

Q3. Passing Objects to Methods

Define a class `Book` with public fields: `string title` and `double price`.
Write a function `void displayBook(Book b)` that prints the book's title and price.
In `main()`, create a `Book` object, assign values, and pass it to `displayBook`.

Q4. Method-Call Stack & Scope of Declarations

What is the output of the following code? (Do not run it – trace the stack frames and scope rules.)

```
#include <iostream>
using namespace std;

int x = 10;   // global

void func() {
    int x = 20;   // local to func
    cout << x << " ";
}

int main() {
    int x = 30;   // local to main
    func();
    cout << x << " ";
    return 0;
}
```

Q5. Argument Promotion and Casting

Write a function `void printDouble(double d)` that prints the value of `d`.
In `main()`, call this function with:

- An integer `5` (implicit promotion)
- A character `'A'` (implicit promotion)
  Then call it again with a `float` value `3.14f` but **explicitly cast** it to `double` using `static_cast<double>`.

Q6. Passing Arrays to Methods

Write a function `int sumArray(int arr[], int size)` that returns the sum of all elements in the array.

In `main()`, declare an integer array of size 6, initialize it with values `{2, 4, 6, 8, 10, 12}`, pass it to the function, and print the returned sum.

Q7. Pass-By-Value vs. Pass-By-Reference

Write two functions:

- `void swapByValue(int a, int b)` – swaps the two parameters locally.
- `void swapByReference(int &a, int &b)` – swaps the actual arguments.
  In `main()`, declare two variables `p = 10` and `q = 20`. Call `swapByValue(p, q)` and print `p` and `q`. Then call `swapByReference(p, q)` and print `p` and `q` again to show the difference.

Q8. Arrays of Objects

Define a class `Student` with public fields: `string name` and `int marks`.

In `main()`, create an **array of 3 Student objects**. Assign names and marks to each object (e.g., "Ali", 85; "Sara", 92; "Usman", 78). Then write a loop to print the name and marks of every student in the array.

Q9. Multidimensional Arrays

Write a function `void printMatrix(int matrix[2][3])` that prints a 2×3 matrix in the following format (each row on a new line, numbers separated by spaces).
In `main()`, initialize a 2×3 array with numbers `1` to `6` and pass it to the function.

Q10. Scope, Static Local, and Pass-by-Reference (Combined)

What is the output of the following code? Trace the variable scopes and the effect of `static`.

```
#include <iostream>
using namespace std;

int global = 1;

void process(int &x) {
    static int staticVar = 0;
    staticVar++;
    x = x + staticVar;
    global = global + x;
}

int main() {
    int a = 5;
    process(a);
    cout << a << " " << global << endl;
    process(a);
    cout << a << " " << global << endl;
    return 0;
}
```